

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- CLI To-Do List Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 4
 - 2.1 Project Implementation Roadmap & Traceability 4
 - 2.2 CLI Task Management Workflow 4
 - 3. Detailed Technical Specifications & Business Rules 5
 - 3.1 Requirements Traceability 5
 - 3.2 Security Rules 7
 - 3.3 Data Models 7
 - 4. Project Governance & Structural Gaps 8
 - 4.1 Structural Gaps 8
 - 4.2 Remediation & Workflow 8
 - 5. Technical & Domain Glossary (Terminology Reference) 8

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The CLI To-Do List Manager is a Python-based command-line application designed for task lifecycle management using a local JSON storage pattern. It implements a service-oriented architecture separating CLI dispatch, business logic, and file I/O to enable independent user story implementation and testing.

1.2 Maturity Assessment

The project is structurally sound with a high health index, though it lacks formal acceptance criteria and explicit security/performance constraints. Given that only one minor task (T017) remains pending and the core infrastructure is validated, the project is READY for final execution.

1.3 Technical Stack

- Python
- pyproject.toml

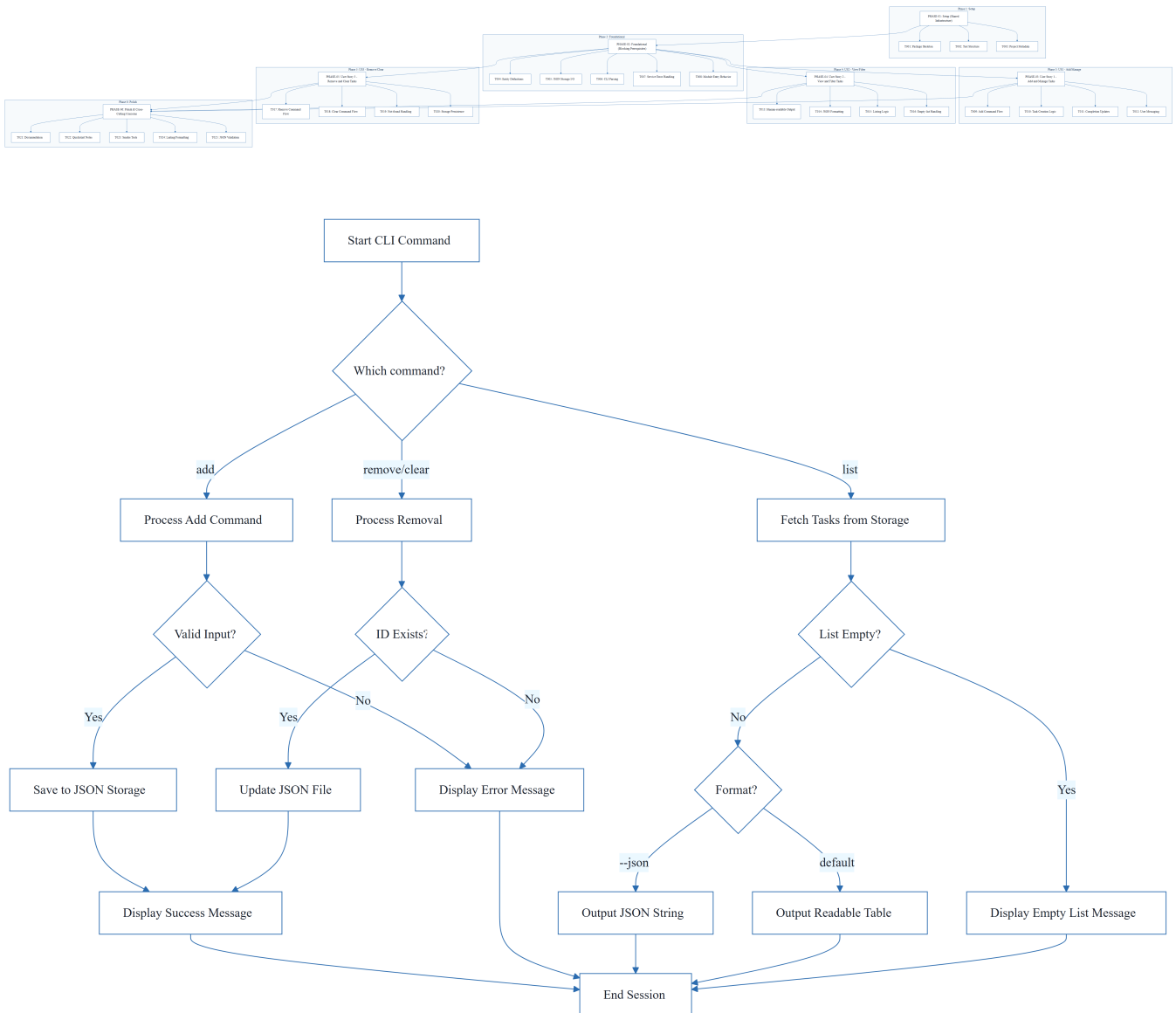
1.4 Architectural Constraints

- Sequential ID assignment for task creation.
- Strict JSON serialization for data persistence.
- Human-readable and `--json` output modes for listing.
- Blocking dependency: Foundational phase must be complete before any User Story implementation.
- Storage path resolution restricted to `~/ .todos.json`.

1.5 Critical Dependencies

- JSON file I/O for persistence in `~/ .todos.json`.
- Sequential dependency: Setup $\rightarrow$ Foundational $\rightarrow$ User Stories $\rightarrow$ Polish.
- Internal data integrity: Task entity serialization helpers in `models.py`.
- CLI entry point configuration in `pyproject.toml`.
- Referential integrity for task removal based on unique IDs.

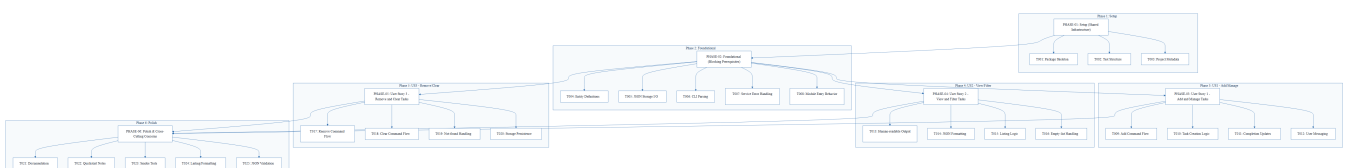
2. Architecture Workflows



& Visual Diagrams

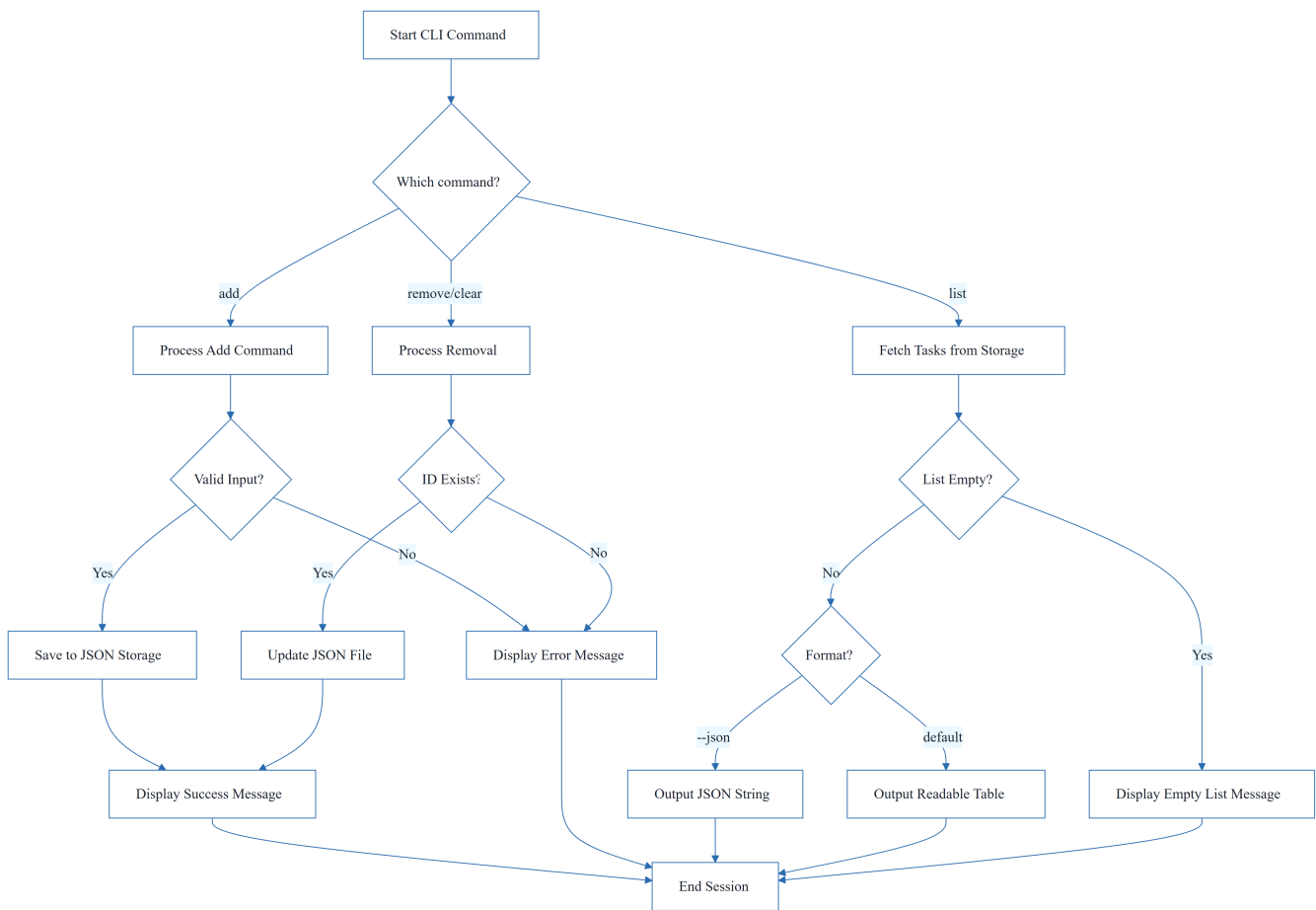
2.1 Project Implementation Roadmap & Traceability

Visualizes the dependency chain between project phases and the specific tasks contained within each phase, ensuring strict identifier preservation.



2.2 CLI Task Management Workflow

Models the business logic for task operations including decision points for input validation and output formatting.



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Source Section	Description	Status
T001	Phase 1: Setup	Create the Python package skeleton in src/todo_manager/init.py, main.py, cli.py, models.py, storage.py, and service.py	completed
T002	Phase 1: Setup	Create the test directory structure in tests/unit/ and tests/integration/	completed
T003	Phase 1: Setup	Add project metadata and tooling entry points in pyproject.toml	completed

ID	Source Section	Description	Status
T004	Phase 2: Foundational	Implement task entity definitions and serialization helpers in <code>src/todo_manager/models.py</code>	completed
T005	Phase 2: Foundational	Implement JSON storage path resolution and file I/O helpers in <code>src/todo_manager/storage.py</code>	completed
T006	Phase 2: Foundational	Implement shared command-line parsing and top-level dispatch in <code>src/todo_manager/cli.py</code>	completed
T007	Phase 2: Foundational	Implement shared service-layer error handling and task collection utilities in <code>src/todo_manager/service.py</code>	completed
T008	Phase 2: Foundational	Define module entry behavior for python <code>-m todomanager</code> in <code>src/todomanager/main.py</code>	completed
T009	Implementation for US1	Implement the add command flow in <code>src/todomanager/cli.py</code> and <code>src/todomanager/service.py</code>	completed
T010	Implementation for US1	Implement task creation, sequential ID assignment, and created at <i>population</i> in <code>src/todomanager/models.py</code>	completed
T011	Implementation for US1	Implement completion updates for existing tasks in <code>src/todo_manager/service.py</code>	completed
T012	Implementation for US1	Add user-facing success and error messages for add and complete operations in <code>src/todo_manager/cli.py</code>	completed
T013	Implementation for US2	Implement human-readable task listing output in <code>src/todo_manager/cli.py</code>	completed
T014	Implementation for US2	Implement <code>--json</code> output formatting in <code>src/todomanager/cli.py</code> using the task serialization helpers in <code>src/todomanager/models.py</code>	completed

ID	Source Section	Description	Status
T015	Implementation for US2	Add listing logic that preserves task order and status fields in <code>src/todo_manager/service.py</code>	completed
T016	Implementation for US2	Handle the empty-list case with a clear message in <code>src/todo_manager/cli.py</code>	completed
T017	Implementation for US3	Implement the remove command flow in <code>src/todomanager/cli.py</code> and <code>src/todomanager/service.py</code>	pending
T018	Implementation for US3	Implement the clear command flow for removing completed tasks in <code>src/todo_manager/service.py</code>	completed
T019	Implementation for US3	Add not-found handling for invalid task IDs in <code>src/todo_manager/cli.py</code>	completed
T020	Implementation for US3	Ensure storage writes persist removals and clears safely in <code>src/todo_manager/storage.py</code>	completed
T021	Phase 6: Polish	Document the CLI usage and storage behavior in <code>README.md</code>	completed
T022	Phase 6: Polish	Add quickstart verification notes and examples in <code>specs/001-cli-todo-manager/quickstart.md</code>	completed
T023	Phase 6: Polish	Run a manual smoke test of add, list, complete, remove, and clear against <code>~/todos.json</code>	completed
T024	Phase 6: Polish	Verify linting and formatting expectations for the source files in <code>src/todo_manager/</code>	completed
T025	Phase 6: Polish	Confirm the generated JSON output remains parseable for the todo list <code>--json</code> path	completed

3.2 Security Rules

No specific security constraints were defined for this CLI tool.

3.3 Data Models

- **Task Entity:** Defined in `src/todo_manager/models.py`.
- **Persistence:** JSON format stored at `~/todos.json`.
- **Attributes:** Includes sequential ID, description, completion status, and `created_at` timestamp.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Missing Section	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	While 'Independent Tests' are mentioned for user stories, formal acceptance criteria for each task are missing.
Security & Performance Constraints	LOW	No specific security or performance constraints were defined for this CLI tool.
Open Questions & Uncertainties	LOW	The document appears complete with no open questions listed.

4.2 Remediation & Workflow

The project follows an incremental delivery model:

1. **Setup $\rightarrow$ Foundational:** Establish core I/O and entity models.
2. **User Story Implementation:** Implement US1 (MVP), then US2, then US3.
3. **Validation:** Each story is validated independently before proceeding.
4. **Polish:** Final smoke tests and documentation.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-02	A synchronization gate ensuring all blocking prerequisites are verified before parallel development of user stories commences.
Foundational	TECHNICAL_STACK	PHASE-02	The shared infrastructure layer containing core entity definitions and I/O helpers that blocks all subsequent feature implementation.

Term	Category	Context Anchor	Project Definition
Goal	BUSINESS_DOMAIN	PHASE-03	The high-level functional objective that a specific user story must satisfy to be considered complete.
ID	BUSINESS_DOMAIN	T010	A sequential alphanumeric token assigned to each task to ensure unique identification within the persistent store.
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage in ~/.todos.json and for machine-readable output.
MVP	BUSINESS_DOMAIN	PHASE-03	The minimum set of functional capabilities, specifically adding and completing tasks, required for the first viable release.
Organization	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The structural grouping of implementation units by user story to maintain independent testability.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of design documents including plan, spec, and data-model required before execution begins.
README	TECHNICAL_STACK	T021	The primary documentation file detailing command-line usage and storage behavior.
Setup	TECHNICAL_STACK	PHASE-01	The initial phase focused on project skeleton creation and pyproject.toml configuration.
T016	TECHNICAL_STACK	T016	The specific implementation unit responsible for handling empty-list scenarios in the user interface.
Tests	TECHNICAL_STACK	T002	The verification suite divided into unit and integration directories to validate system behavior.
population in	BUSINESS_DOMAIN	T010	The process of assigning a timestamp to the created_at field during task instantiation.

Term	Category	Context Anchor	Project Definition
todo clear	BUSINESS_DOMAIN	T018	The operational command that purges all entries marked as completed from the storage.
todo list	BUSINESS_DOMAIN	T013	The operational command that retrieves and displays all stored entries in human-readable or machine-parseable formats.
■■ CRITICAL	TECHNICAL_STACK	PHASE-02	A high-priority constraint indicating that no subsequent work can proceed until the current phase is fully validated.